SELinux the Easy Way

A Practical Guide for QA Engineers

Andrea Manzini

Software Quality Engineer @ SUSE

May 2026

📋 Agenda

0. SELinux Basics and QA Context
1. Modes & Access Control (DAC + MAC)
2. Policies, Rules & Classes
3. Labels: Understand, Inspect, Fix
4. Booleans, Ports, and File Contexts
5. QA Testing Guidelines
6. Troubleshooting Denials
7. RKE2 and Storage
8. Live Demos (if time permits)
9. QA Cheat Sheet + References

🤔 What's SELinux?

SELinux (Security-Enhanced Linux) is a security architecture for Linux systems that provides Mandatory Access Control (MAC).

- **MAC vs. DAC:** Unlike traditional Linux permissions (Discretionary Access Control) where users control their files, MAC enforces policies set by the administrator.
- **Default Deny:** If there isn't an explicit rule allowing an action, it is blocked.
- Originally developed by the NSA, now integrated into the mainline Linux kernel.
- Implemented via Linux Security Modules (LSM).

Why SELinux?

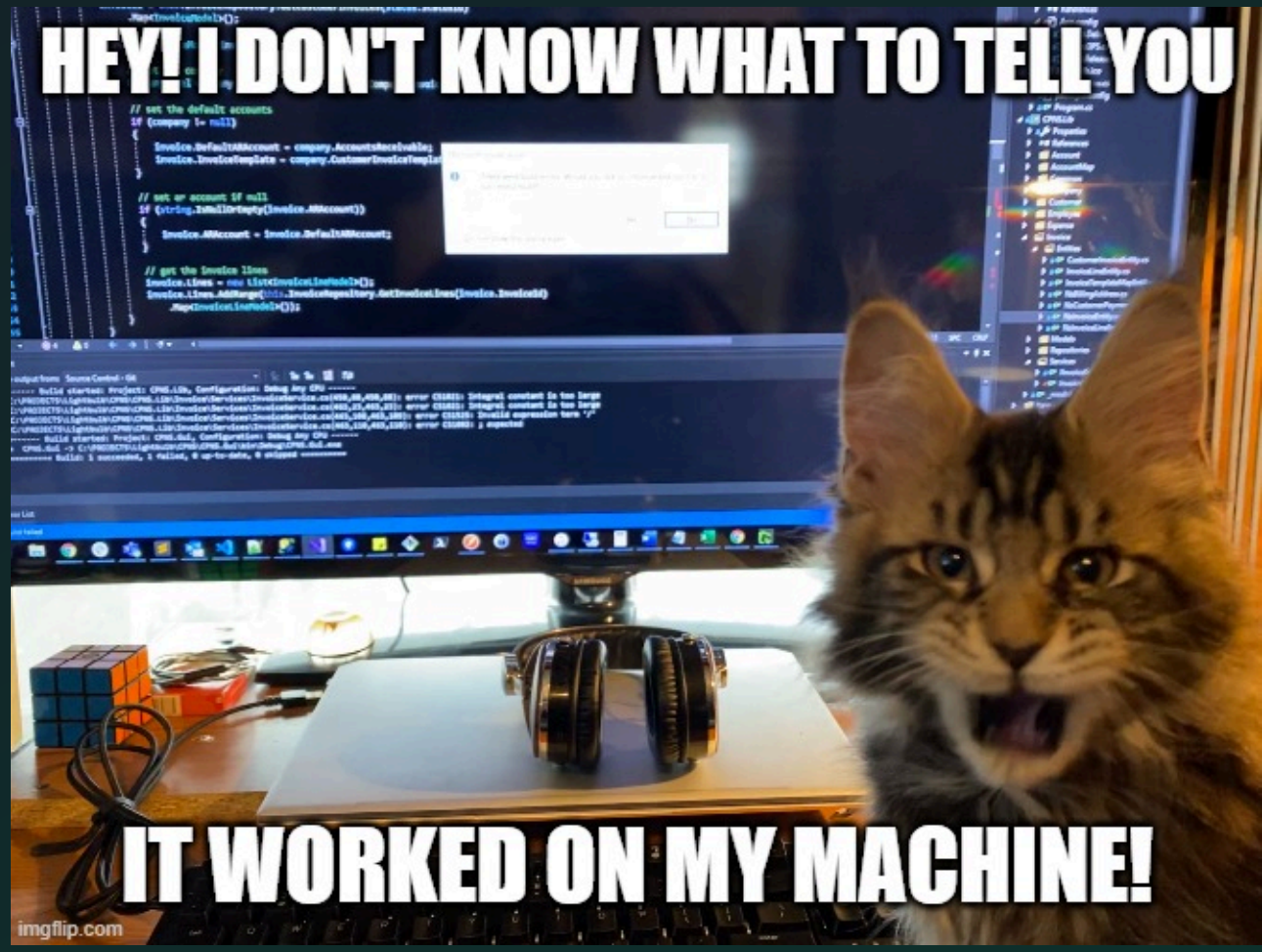
- **Containment:** If a service (like a web server) is compromised, SELinux prevents the attacker from accessing the rest of the system or other services.
- **Privilege Escalation Prevention:** Even an exploit running as `root` can be contained if SELinux policies restrict what that specific `root` process can do.
- **Zero-Day Protection:** Helps mitigate the impact of unknown vulnerabilities by enforcing strict behavioral boundaries.
- **Compliance:** Often required by security standards and enterprise environments.

🎯 Why QA Cares About SELinux

- SELinux is the **default MAC** on SLES 16, Leap 16, and Tumbleweed
- Customers run SELinux in **Enforcing** mode — so must QA
- SELinux denials look like permission errors — easy to misdiagnose
- A proper bug report with SELinux context saves developers **hours**

Happens every time

> "It works on my machine" — because you had SELinux disabled.



SELinux Modes

Mode	Behavior	When to use
Enforcing	Blocks + logs denials	Always (production & testing)
Permissive	Logs but does NOT block	Diagnosing SELinux issues only
Disabled	SELinux not initialized	Never (requires reboot + full relabel to re-enable)

```
$ sestatus
```

```
$ getenforce                    # Check current mode
Enforcing

$ sudo setenforce 0             # Switch to Permissive (temporary)
$ sudo setenforce 1             # Back to Enforcing

# Permanent: edit /etc/selinux/config and reboot
SELINUX=enforcing
```

⚙️ How SELinux Works (see link for picture)

SELinux acts as a gatekeeper between a **Subject** (usually a process) and an **Object** (a file, directory, port, etc.).

1. **Subject Request:** A process requests an action (e.g., `read`) on an object (e.g., a file).
2. **DAC Check:** Standard Linux permissions (Owner/Group/Others) are checked first. If this fails, access is denied immediately.

but there's more ...

⚙️ How SELinux Works

3. **MAC Check:** If DAC passes, SELinux steps in. It looks at the **scontext** (source context — the process) and the **tcontext** (target context — the file).
4. **Policy Evaluation:** The SELinux policy is consulted to see if there is an explicit rule allowing the scontext to perform the requested action on the tcontext.
5. **Decision:** Allowed if a rule exists, blocked otherwise (Default Deny).

see also This quick intro

📖 Policies: The Rulebook

A **policy** is the set of rules that defines what each domain (process type) is allowed to do.

Policy types (set in `/etc/selinux/config`):

Policy	Description	Use case
targeted	Only known services are confined, everything else runs unconfined	Default on SLES 16 / Tumbleweed / Leap 16
minimum	Like targeted but fewer services confined	Minimal environments (not shipped on SUSE)
mls	Full Multi-Level Security	Government/military only

Policies are modular — built from independent `.pp` modules:

```
# List loaded policy modules
$ sudo semodule -l | head
abrt
account-utils
accountsd
acct
afs

# Install a custom module
$ sudo semodule -i my_fix.pp

# Remove a module
$ sudo semodule -r my_fix
```

SLES 16 ships **440+ policy modules** out of the box (based on Fedora upstream policy with SUSE-specific patches). Custom modules are how you extend the policy. `audit2allow -M` generates them.

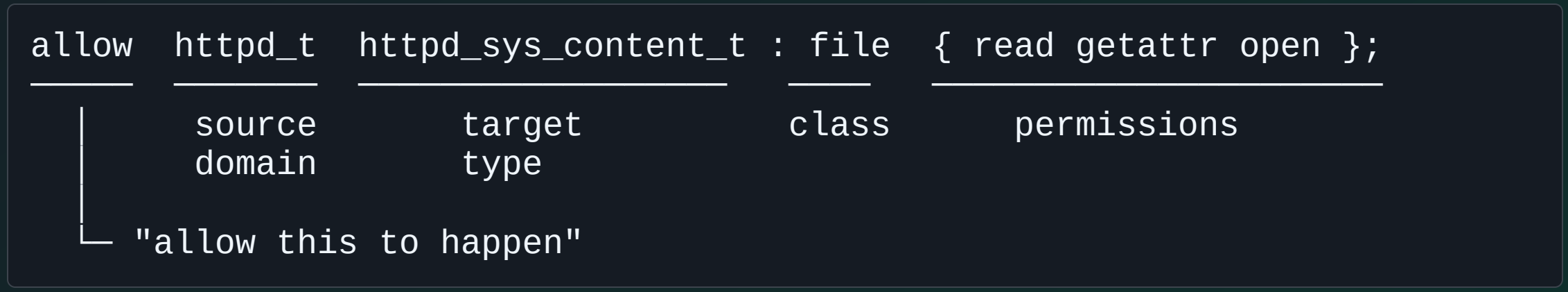
```
# See specific module source (if selinux-policy-devel installed)
sudo zypper install selinux-policy-devel
ls /usr/share/selinux/devel/

# Extract a loaded module to CIL (text) format
sudo semodule -E ftpd              # writes ftpd.cil in cwd
sedismod ftpd.pp                   # disassemble .pp binary policy

# Or use sesearch to query rules directly
sudo zypper install setools-console
sesearch --allow -s ftpd_t           # all allow rules for ftpd
sesearch --allow -t etc_t            # all rules targeting etc_t
```

🧩 Policy Rules: A Concrete Example

A Type Enforcement (TE) rule looks like this:



Read it as: "A process running in the `httpd_t` domain is allowed to `read`, `getattr`, and `open` files labeled `httpd_sys_content_t`."

🧩 Policy Rules: A Concrete Example

You can query existing rules with `sesearch`:

```
# What can httpd_t read?
$ sesearch --allow -s httpd_t -t httpd_sys_content_t -c file
allow httpd_t httpd_sys_content_t:file { getattr ioctl lock map open read };

# What can write to files labeled httpd_log_t?
$ sesearch --allow -t httpd_log_t -c file -p write
allow httpd_t httpd_log_t:file { append create write ... };
```

If no rule exists → **denied**. That's the "default deny" principle.

📏 SELinux Rule Types

Rule Types (how to control):

- `allow` — permits action
- `type_transition` — when X runs Y, become Z domain
- `type_change` — process changes domain
- `dontaudit` — suppresses logging (for noisy but safe denials)
- `auditallow` — forces logging of allowed actions

Example: `type_transition init_t sshd_exec_t:process sshd_t;` = "when init executes sshd binary, transition to sshd_t domain"

🎓 SELinux Classes

Common Object Classes (what can be controlled):

- `file`, `dir` — filesystem objects
- `process`, `domain` — running processes
- `tcp_socket`, `udp_socket` — network sockets
- `capability` — system capabilities (mknod, net_admin, etc.)
- `fd` — file descriptors
- `dir_search_perms`, `open`, `read`, `write` — specific permissions

📂 Where Do Policies Live? (on SUSE)

```
/etc/selinux/
├── config                          # SELINUX=enforcing, SELINUXTYPE=targeted
└── targeted/
    ├── policy/policy.35            # compiled binary policy (loaded by kernel)
    ├── contexts/                   # default contexts for logins, services, etc.
    └── modules/                    # installed policy modules (recently migrated here)

/usr/share/selinux/targeted/        # source .pp modules shipped by packages
```

Key packages (zypper):

Package	Contents
`selinux-policy`	Base policy, file contexts, core types
`selinux-policy-targeted`	The targeted policy variant (440+ modules)
`container-selinux`	Policy for Podman/Docker containers
`policycoreutils-python-utils`	`semanage`, `audit2allow`, `audit2why`

SUSE-specific: policy modules recently migrated from `/var/lib/selinux` to `/etc/selinux` so they're covered by BTRFS snapshots and rollbacks.

🏷️ What is a Label?

Every object in the system has a **security context** (label).

A label is made of 4 parts: user, role, type, level

```
system_u : object_r : httpd_sys_content_t : s0
  user       role              type          level
```

File labels are stored as **extended attributes (xattr)** on the inode:

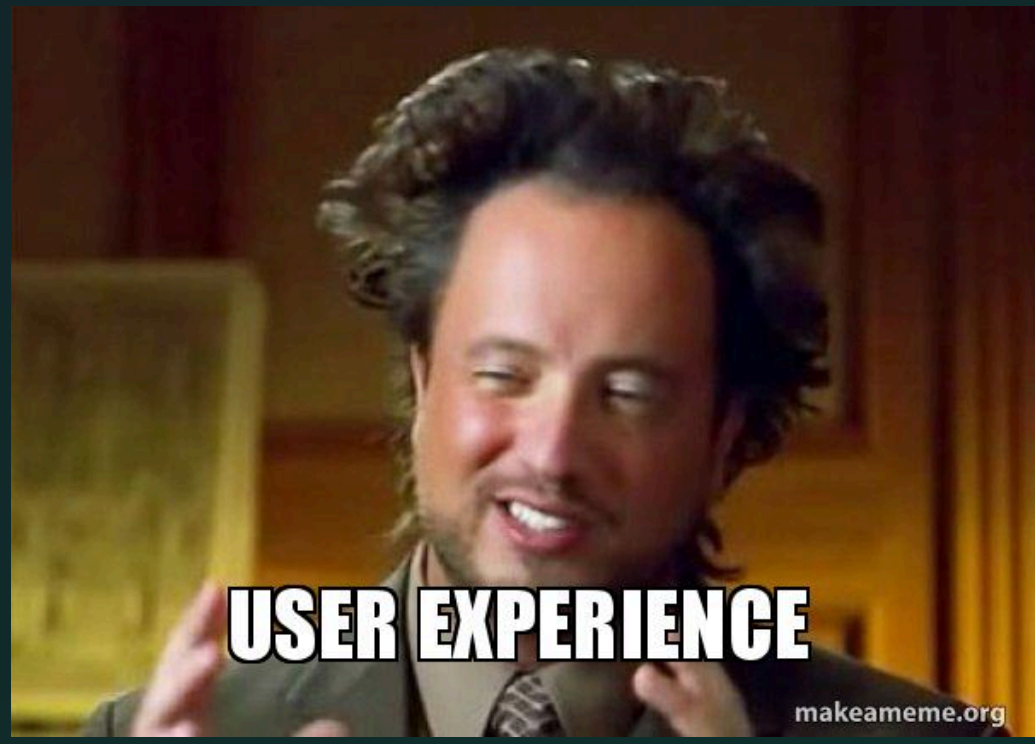
```
$ getfattr -d -m . /var/www/html/index.html
# security.selinux="system_u:object_r:httpd_sys_content_t:s0"
```

for that reason SELINUX needs a filesystem which supports xattrs.

🔍 Label Fields Explained : User

User (`system_u` , `unconfined_u` , `staff_u`)

- Used by **RBAC (Role-Based Access Control)** policies.
- Maps Linux users to SELinux users. Controls which roles are available to a user.
- `system_u` = system daemons, `unconfined_u` = regular users (no restrictions)



🔍 Label Fields Explained : Role

Role (`system_r` , `object_r` , `unconfined_r`)

- Also used by **RBAC**.
- Bridges users to types — dictates which domains (types) a user can transition into.
- `system_r` = system processes, `object_r` = files/passive objects

🔍 Label Fields Explained: Type

Type (`httpd_t` , `user_home_t` , `tmp_t`)

- Used by **TE (Type Enforcement)** policies.
- **The field you'll work with 95% of the time.** In "targeted" policy, almost all rules check this.
- Processes have a **domain** (`httpd_t`), files have a **type** (`httpd_sys_content_t`).
- Rule format: `allow <domain> <type> : <class> { <permissions> };`

🔍 Label Fields Explained: Level

Level (`s0` , `s0:c1,c2`)

- Used mostly by **MLS/MCS (Multi-Level / Multi-Category Security)** policies and kubernetes.
- Defines sensitivity and categories. Crucial for **Container isolation** (e.g., Podman gives each container a unique category like `c1,c2`).

👀 Viewing Labels

```
# Files
$ ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html

# Processes
$ ps -eZ | grep nginx
system_u:system_r:httpd_t:s0     1234 ?  00:00:00 nginx

# Your own context
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Rule: if a process in domain `X_t` tries to access a file of type `Y_t`, SELinux checks the policy for an `allow X_t Y_t` rule.

✏️ Changing Labels: 2 Approaches

There are **two ways** to change a file's SELinux label:

1. **Temporary (xattr)**: use `chcon` to modify the label directly on the file/inode
2. **Permanent (rules)**: define a local labeling rule so the label persists

🔧 Method 1: Temporary with `chcon`

`chcon` modifies the **xattr directly** on the file's inode — changes the label **right now**, but doesn't update the permanent rule:

```
$ chcon -t httpd_sys_content_t /srv/myfile.txt
$ ls -Z /srv/myfile.txt
system_u:object_r:httpd_sys_content_t:s0  /srv/myfile.txt
```

Problem: When `restorecon` runs, it consults `file_contexts.local`, finds no rule for your file, and **reverts the label back to default**. Your change vanishes.

Use case: Quick testing only.

🔧 Method 2: Permanent with `semanage` + `restorecon`

Define the rule **once**, `restorecon` applies it **forever**:

```
# Add the rule
$ sudo semanage fcontext -a -t httpd_sys_content_t '/srv/myapp(/.*)?'

# Apply to existing files
$ sudo restorecon -Rv /srv/myapp
```

The rule is stored in `/etc/selinux/targeted/contexts/files/file_contexts.local`:

```
/srv/myapp(/.*)?  --  system_u:object_r:httpd_sys_content_t:s0
```

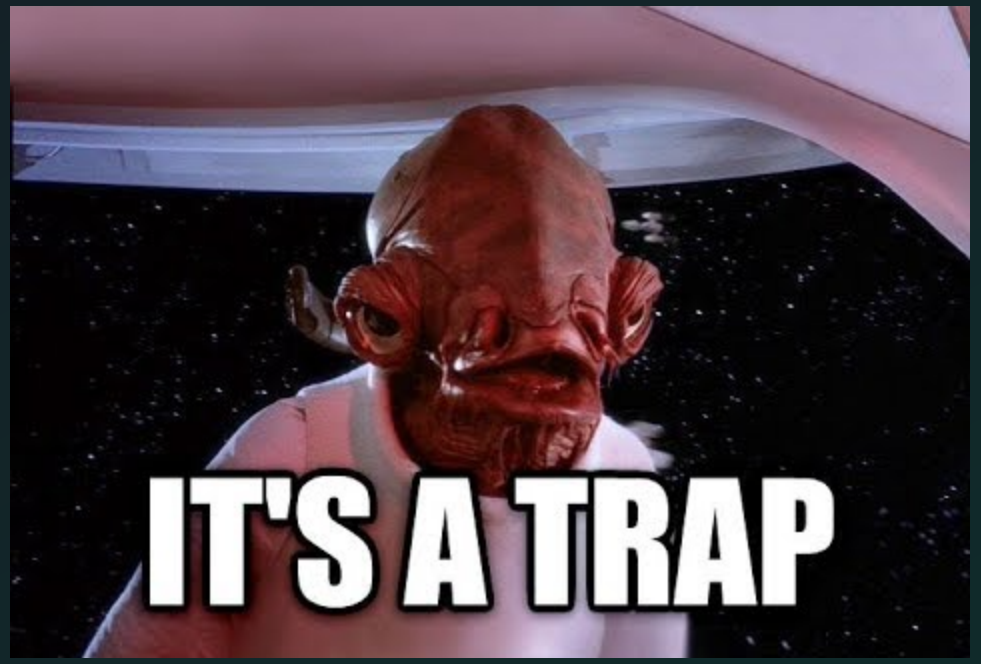
Use case: Production fixes. Label persists across relabels, package updates, `mv`, etc.

QA tips

- Always use `semanage` + `restorecon` in test process. `chcon` changes silently vanish after `restorecon -R` or package updates.
- Propagate the changes to upstream

⚠️ The mv/cp Trap

```
# when a script creates a test config in temp directory
$ echo "server {}" > /tmp/mysite.conf
$ ls -Z /tmp/mysite.conf
unconfined_u:object_r:user_tmp_t:s0  /tmp/mysite.conf
```



⚠️ The mv/cp Trap

COPY creates a new file inode → file will be labeled by context rules

```
$ cp /tmp/mysite.conf /etc/nginx/conf.d/
$ ls -Z /etc/nginx/conf.d/mysite.conf
unconfined_u:object_r:httpd_config_t:s0     # correct!
```

MOVE keeps original label

```
$ mv /tmp/mysite.conf /etc/nginx/conf.d/
$ ls -Z /etc/nginx/conf.d/mysite.conf
unconfined_u:object_r:user_tmp_t:s0         # WRONG! nginx can't read this
```

Fix: `restorecon -v /etc/nginx/conf.d/mysite.conf`

🪲 QA Angle: Labels in Bug Reports

When filing a bug where SELinux might be involved, **always include**:

```
# 1. Label of the file/directory that failed
ls -Z /path/to/file

# 2. Label of the process that got denied
ps -eZ | grep <process_name>

# 3. What label the file SHOULD have
restorecon -n -v /path/to/file

# 4. Current SELinux mode
getenforce
```

A bug report that says "permission denied" without this info is **incomplete**.

Booleans: Policy Switches

Modify policy *behavior* **without writing custom rules**.

```
# List all booleans related to httpd
$ getsebool -a | grep httpd
httpd_can_network_connect --> off
httpd_can_sendmail --> off
httpd_enable_homedirs --> off

# Allow Apache to make network connections
$ sudo setsebool -P httpd_can_network_connect on
```

The `-P` flag = persistent across reboots. Without it, the change is lost on restart.

What's a Boolean? Example

A boolean is a **policy toggle**. It switches entire sets of allow rules on/off without editing the policy.

```
httpd_can_network_connect = off (default)
  ↓
  Policy has NO rules: allow httpd_t any_domain:tcp_socket { connect ... };
  nginx CANNOT make outbound connections

httpd_can_network_connect = on
  ↓
  Same policy now includes those rules
  nginx CAN make outbound connections
```

Real use case: Your web app needs to call a backend API on `http://db.local:5432`. Set the boolean, deny is gone.

What about network ?

SELinux controls which processes can **bind** to which ports.

Port labels are stored in the SELINUX database (managed via `semanage port`):

```
$ sudo semanage port -l | grep http
http_cache_port_t              tcp      8080, 8118, 8123, 10001-10010
http_cache_port_t              udp      3130
http_port_t                    tcp      80, 81, 443, 488, 8008, 8009, 8443, 9000
http_port_t                    udp      80, 443
pegasus_http_port_t            tcp      5988
pegasus_https_port_t           tcp      5989
```

🔌 Port Labeling

```
# See what ports SSH is allowed to use
$ sudo semanage port -l | grep ssh
ssh_port_t                     tcp      22

# Move SSH to port 2222 → it will FAIL to start
# Fix: add the port to the policy
$ sudo semanage port -a -t ssh_port_t -p tcp 2222

# Verify
$ sudo semanage port -l | grep ssh
ssh_port_t                     tcp      2222, 22
```

Same applies to any service using a non-standard port.

🔧 From binary to port 🔌

- When the system boots and systemd launches the SSH daemon, the executable file (usually `/usr/sbin/sshd`) is read.
- The file itself has a context label of `sshd_exec_t`:

```
$ ls -Z $(which sshd)
system_u:object_r:sshd_exec_t:s0 /usr/sbin/sshd*
```

- SELinux policy dictates a domain transition: when an init system executes a file labeled `sshd_exec_t`, the resulting running process must transition into the restricted `sshd_t` domain.

```
$ sudo sesearch -T -s init_t -t sshd_exec_t -c process
type_transition init_t sshd_exec_t:process sshd_t;
```

🔧 From binary to port 🔌

- The `sshd` process tries to bind and listen on port 22. The port is identified as `ssh_port_t`:

```
$ sudo semanage port -l | grep ssh
ssh_port_t                     tcp      22
```

- A *permit* rule states that processes of domain `sshd_t` can bind to port type `ssh_port_t`, and send/receive bytes from/to that port.

```
$ sudo sesearch --allow -s sshd_t -t ssh_port_t | grep sshd_t
```

📁 File Context Rules

The **permanent** way to manage labels for custom paths:

```
# Tell SELinux: everything under /srv/myapp is web content
$ sudo semanage fcontext -a -t httpd_sys_content_t '/srv/myapp(/.*)?'

# Apply the rule to existing files
$ sudo restorecon -Rv /srv/myapp
Relabeled /srv/myapp from unconfined_u:object_r:default_t:s0
  to unconfined_u:object_r:httpd_sys_content_t:s0

# Verify
$ ls -Z /srv/myapp/
system_u:object_r:httpd_sys_content_t:s0 index.html
```

✅ QA Recap: Testing with SELinux

Rules for QA test environments:

1. **Always test in Enforcing mode** — matches customer environments
2. Use Permissive **only** to diagnose, then switch back immediately
3. Never put `setenforce 0` in test setup scripts
4. After installing/upgrading a package, run `restorecon -R` on its paths
5. If a test fails with "Permission denied", check SELinux **before** filing a DAC bug

🔎 Where to Look for Denials

```
# The audit log (primary source)
$ sudo ausearch -m AVC -ts recent

# Or directly
$ sudo grep "avc:  denied" /var/log/audit/audit.log

# Via journald
$ sudo journalctl -t audit --since "10 minutes ago" | grep AVC
```

No denials showing? Check:

- Is `auditd` running? (`systemctl status auditd`)
- Is SELinux actually in Enforcing? (`getenforce`)
- Some denials are "dontaudit" — use `semodule -DB` to disable dontaudit rules and rebuild policy (re-enable with `semodule -B`)

🔬 Anatomy of a Denial

```
type=AVC msg=audit(1716000000.123:456): avc:  denied  { read }
  for  pid=1234 comm="nginx" name="mysite.conf"
  dev="sda1" ino=5678
  scontext=system_u:system_r:httpd_t:s0
  tcontext=unconfined_u:object_r:user_tmp_t:s0
  tclass=file permissive=0
```

Field	Meaning
`{ read }`	Which action was denied
`comm="nginx"`	Which program tried
`scontext=...httpd_t...`	Process domain (subject)
`tcontext=...user_tmp_t...`	File type (target)
`tclass=file`	Object class

⚡ Quick Check: is it a SELinux issue?

```
# Step 1: Something fails. Is SELinux blocking it?
$ sudo setenforce 0          # Permissive mode

# Step 2: Retry the operation
$ sudo systemctl restart nginx
# If it works now → SELinux was blocking it

# Step 3: IMMEDIATELY switch back
$ sudo setenforce 1

# Step 4: Find the denial
$ sudo ausearch -m AVC -ts recent
```

Never leave a system in Permissive mode after testing. Automate: set a timer or use `at` to switch back.

🔧 audit2why & audit2allow

```
# WHY was it denied?
$ sudo ausearch -m AVC -ts recent | audit2why
Was caused by: Missing type enforcement (TE) allow rule.
  Allow rules: allow httpd_t user_tmp_t:file read;

# WHAT rule would fix it?
$ sudo ausearch -m AVC -ts recent | audit2allow
#============= httpd_t ==============
allow httpd_t user_tmp_t:file read;

# Generate a loadable policy module
$ sudo ausearch -m AVC -ts recent | audit2allow -M my_nginx_fix
$ sudo semodule -i my_nginx_fix.pp
```

Warning: `audit2allow` may suggest overly broad rules. Always check what it proposes before loading.

🚨 setroubleshoot / sealert

Human-readable analysis of denials:

```
$ sudo sealert -a /var/log/audit/audit.log

SELinux is preventing nginx from read access on the file mysite.conf.

*****  Plugin restorecon (99.5 confidence) suggests   ***************
If you want to fix the label:
  restorecon -v /etc/nginx/conf.d/mysite.conf

*****  Plugin catchall (1.49 confidence) suggests   ******************
If you want to allow httpd_t to read user_tmp_t files:
  audit2allow -M mypol
```

`sealert` ranks suggestions by confidence — start from the top.

🐳 Container Volume Isolation (MCS)

SELinux uses **MCS** (Multi-Category Security) to isolate containers.

- Each container gets unique category pair: `s0:c123,c456`
- Prevents containers from accessing each other's volumes
- Even if running as same UID

Example Pod security context:

```
spec:
  securityContext:
    seLinuxOptions:
      level: "s0:c0.c1023"  # allow broad category range
```

Kubernetes 1.27+: Efficient mount-time labeling — no recursive `chcon` on large volumes. See K8s efficient relabeling blog.

🔧 Storage Backend Issues & Fixes

Backend	Issue	Fix
Longhorn	iSCSI `dac_override` denials (container-selinux v2.189.0+)	Load custom module: `echo '(allow iscsid_t self (capability (dac_override)))' > fix.cil && semodule -vi fix.cil` KB
local-path-provisioner	SELinux blocks hostPath access	`chcon -t container_file_t -R /opt/local-path-provisioner` Issue
NFS volumes	Container can't mount NFS	`setsebool -P virt_use_nfs on`

📚 RKE2 + SELinux References

Official docs:

- RKE2 SELinux Documentation
- SUSE RKE2 SELinux Guide
- Kubernetes Security Contexts

Known issues:

- RKE2 Known Issues — SELinux module upgrade problems
- Downstream cluster SELinux setup

QA testing notes:

- Always test with `selinux: true` in RKE2 config
- Check volume mount denials: `ausearch -m AVC | grep mount`

🧪 Demo Setup

Prerequisites: openSUSE Tumbleweed with `podman` installed.

```
# From this repo's directory:
$ ./selinux-lab.sh

# This will:
# 1. Create a Tumbleweed distrobox named "selinux-lab"
# 2. Install SELinux tools, nginx, audit utilities
# 3. Copy a sample audit log for offline demos

# Enter the lab:
$ podman start -ai selinux-lab
```

All demo commands in the following slides can be run inside this container. Note: actual enforcement requires a system with SELinux enabled in the kernel (Tumbleweed, SLES 16).

Demo: nginx Can't Read Its Config

```
# 1. Create a config file in /tmp
echo 'server { listen 8080; root /usr/share/nginx/html; }' \
  > /tmp/demo.conf

# 2. Move it (not copy!) to nginx config dir
mv /tmp/demo.conf /etc/nginx/conf.d/

# 3. Check the label — it's WRONG
ls -Z /etc/nginx/conf.d/demo.conf
# → user_tmp_t (should be httpd_config_t)

# 4. On a real enforcing system: nginx would fail to start
#    Check the audit log for the AVC denial

# 5. Fix it
sudo restorecon -v /etc/nginx/conf.d/demo.conf
# → Relabeled to httpd_config_t

# 6. Verify
ls -Z /etc/nginx/conf.d/demo.conf
```

Demo: Service on a Non-standard Port

```
# 1. Check which ports httpd_t can bind to
sudo semanage port -l | grep http_port
# → http_port_t    tcp    80, 81, 443, 488, ...

# 2. Try to use port 8888 → SELinux would block it
#    The AVC denial would show:
#    denied { name_bind } for ... scontext=...httpd_t...

# 3. Add port 8888 to the allowed list
sudo semanage port -a -t http_port_t -p tcp 8888

# 4. Verify
sudo semanage port -l | grep http_port
# → http_port_t    tcp    8888, 80, 81, 443, ...

# 5. Now the service can bind to port 8888
```

📋 QA Cheat Sheet

Task	Command
Check SELinux mode	`getenforce`
View file labels	`ls -Z /path/to/file`
View process labels	`ps -eZ | grep <name>`
Find recent denials	`ausearch -m AVC -ts recent`
Explain a denial	`ausearch -m AVC -ts recent | audit2why`
Fix a wrong label	`restorecon -v /path/to/file`
Check expected label	`restorecon -n -v /path/to/file`
Add permanent label rule	`semanage fcontext -a -t <type> '/path(/.*)?'`
Toggle boolean	`setsebool -P <bool> on`

🙏 Thank You!

Questions?

Andrea Manzini

Software Quality Engineer @ SUSE

Lab setup & slides:
https://github.com/ilmanzo/suse_presentations